웹 원문 스냅샷 | https://github.com/robrosinc/igris_c_sdk_public

접근 시각: 2026-10-16T05:21:34.542275+00:00

main

BranchesTags

Go to file

Code

Open more actions menu

Latest commit

History

1 Commit

1 Commit

Folders and files

NameName

Last commit message

Last commit date

cyclonedds

cyclonedds

dist

dist

python

python

ros2

ros2

.gitattributes

.gitattributes

.gitignore

.gitignore

AMENT_IGNORE

AMENT_IGNORE

COLCON_IGNORE

COLCON_IGNORE

NOTICE

NOTICE

README.md

README.md

View all files

Repository files navigation

IGRIS-C SDK

C++ + Python SDK for the IGRIS-C robot, layered on top of Cyclone DDS. This repository ships the public release artifacts split by language / runtime so each use case stays self-contained.

Requirements

Tool

Version

OS

Linux x86_64 (Ubuntu 24.04, ROS 2 Jazzy)

CMake

≥ 3.14

Compiler

C++17 (GCC 11+ / Clang 14+)

Python

3.12 (Python bindings + ROS 2 nodes)

Runtime / build dependencies

sudo apt install libssl-dev # required

sudo apt install libglfw3-dev libgl1-mesa-dev # for the GUI examples

sudo apt install libopencv-dev # for the sensor viewers

The sensor viewers (cyclonedds_sensor_viewer, ros_sensor_viewer, python/sensor_viewer.py) are the only things here that use OpenCV — the SDK library itself does not. Configure fails if it is missing, rather than quietly dropping the target; build everything else without it with:

cmake -B build -S . -DIGRIS_C_BUILD_VIEWERS=OFF # cyclonedds/

colcon build --cmake-args -DIGRIS_C_BUILD_VIEWERS=OFF # ros2/

Layout

igris_c_sdk_public/

├── README.md

├── NOTICE

├── dist/ # Python wheel (igris_c_sdk-*.whl)

├── cyclonedds/ # Native C++ SDK (uses bundled Cyclone DDS directly)

│ ├── include/igris_c_sdk/ # Public headers

│ ├── lib/ # libigris_c_sdk_lib.a + CMake package config

│ ├── thirdparty/ # Bundled Cyclone DDS headers + static libs

│ ├── licenses/, LICENSE, NOTICE

│ ├── CMakeLists.txt # Examples CMake project (links the SDK)

│ ├── cyclonedds_spdp.xml # DDS config, multicast discovery (start here)

│ ├── cyclonedds_peers.xml # DDS config, unicast — when multicast is blocked

│ ├── *.cpp + *.md # Example sources + per-example notes

│ └── imgui.ini

├── python/ # Python examples + reproducible env

│ ├── pixi.toml, pixi.lock # User-side pixi environment (Python 3.12 + pip)

│ └── *.py, client.ipynb

└── ros2/ # ROS 2 ament package (msg/srv + example nodes)

├── README.md

├── fastdds_peers.xml # Unicast discovery, for when multicast is blocked

├── setup_client_network.sh # Host socket/reassembly tuning for image streams

└── src/

└── igris_c_sdk_public/ # The ROS 2 package. Copy ros2/src/igris_c_sdk_public/

# (this leaf) into your colcon workspace src/

Each top-level subtree is independent — pick the one matching your runtime.

Note that the tree root and the ROS 2 package share the name

igris_c_sdk_public. Only the leaf is a package: the root holds cyclonedds/,

python/ and dist/ alongside it and is not a colcon workspace, so it carries

COLCON_IGNORE. Placing the root in a workspace src/ builds nothing and says

nothing — colcon does not report directories it skipped. Copy the leaf, or build

in place from ros2/ (see ros2/README.md).

Connecting to the robot

Before running any example, the SDK client must join the robot's DDS network. Three things must match the robot.

Robot namespace

Every DDS topic is resolved under a per-unit robot namespace of the form igris_c_⟨robot_id⟩ — the fixed

prefix igris_c_ plus the robot's configured suffix (e.g. igris_c_IG01). You pass the bare topic (lowstate) and the SDK lays out the

wire name as ⟨namespace⟩/⟨topic⟩. Pass the namespace (and the DDS domain id, which mirrors ROS_DOMAIN_ID) to the client:

ChannelFactory::Instance()->Init(domain_id, "igris_c_⟨robot_id⟩"); // C++

igc_sdk.ChannelFactory.Instance().Init(domain_id, "igris_c_⟨robot_id⟩") # Python

The C++ examples take both as arguments: ./cyclonedds_low_level_control ⟨domain_id⟩ igris_c_⟨robot_id⟩.

The development PC and the robot must use the same namespace + domain id.

CYCLONEDDS_URI (network interface)

DDS discovery needs a CycloneDDS XML describing the network interface. Two samples are bundled in

cyclonedds/, and they differ only in how participants find each other:

File

Discovery

Use when

cyclonedds_spdp.xml

multicast (default)

client and robot on the same segment — start here

cyclonedds_peers.xml

unicast to a listed address

multicast is blocked (Wi-Fi robot, wired client)

Edit <NetworkInterface name="..."> to the interface on THIS machine that reaches the robot (list with ip link, or set autodetermine="true"); cyclonedds_peers.xml additionally needs the robot's address in <Peer address="...">. Then export it before running anything:

```
# from cyclonedds/ (where both files live)
export CYCLONEDDS_URI="file://$PWD/cyclonedds_spdp.xml"
export ROS_DOMAIN_ID=0 # for ROS 2 nodes (match the domain)
```

If CYCLONEDDS_URI is unset or the interface is wrong, participant discovery fails and no topics appear.

Topic layout

The robot serves ONE lane, and its igris_c.network.transport decides both the lane and — on the cyclonedds lane — how topic names are laid out on the wire. Pass the matching layout to the examples with --dds-topic-naming:

| Robot igris_c.network.transport | Example flag | Wire name for topic lowstate |
|---|---|---|
| cyclonedds_native (default) | --dds-topic-naming native (the default) | <ns>/lowstate |
| cyclonedds_ros_compatible | --dds-topic-naming ros_compatible | rt/<ns>/lowstate |
| ros | not applicable — use the ros2/ examples, where rmw fixes the layout | rt/<ns>/lowstate |

```
./cyclonedds_low_level_control 0 igris_c_<robot_id> --dds-topic-naming ros_compatible # C++
python3 client.py --domain-id 0 --namespace igris_c_<robot_id> --dds-topic-naming ros_compatible
```

The cyclonedds and Python examples take the flag; the ROS 2 examples do not, because rmw always prepends rt/. A mismatch is silent — the two sides resolve different topic strings and simply never discover each other, with no error on either end. If the robot is on cyclonedds_native (the shipping default) you can leave the flag off.

cyclonedds_ros_compatible aligns topic names only — it does not give you ROS 2 services. On either cyclonedds layout the robot serves its services as request/response topic PAIRS

(see High-level services), so ros2 service list shows
nothing and an rclcpp service client cannot call them. A rclcpp node can subscribe to the
pub/sub topics, but anything that needs a service — torque on/off, PDU init, hand init,
control-mode switches — has to go through the SDK client on this lane. Real ROS 2 services
require the robot on transport: ros and the ros2/ examples.

When discovery still finds nothing

Discovery relies on the multicast group 239.255.0.1, and an access point does not forward it from
its wireless clients to the wired segment. So a robot reachable ONLY over Wi-Fi is invisible to a
wired client even on the same subnet — a successful ping says nothing about multicast. A robot
that also has a working wired link is fine: multicast reaching it in one direction is enough for
discovery to complete, and everything after that is unicast. The symptom is an empty
ros2 topic list, or one that shows only a few of the robot's topics.

On the ROS 2 lane, ros2/fastdds_peers.xml works around it by making discovery go out unicast;
see ros2/README.md. Putting the
robot on wired Ethernet alongside the client removes the need for it entirely.

Quick start

Native C++ (Cyclone DDS directly)

```
cd cyclonedds
mkdir build && cd build
cmake .. && make -j"$(nproc)"
./cyclonedds_low_level_control 0 igris_c_〈your_robot_id〉
```

See cyclonedds/README.md for the full list
of examples and how to override the SDK path.

To consume the SDK from your own CMake project, point find_package
at the bundled config:

```
set(igris_c_sdk_DIR "〈deliverable〉/cyclonedds/lib/cmake/igris_c_sdk")
find_package(igris_c_sdk REQUIRED)
target_link_libraries(your_target PRIVATE igris_c_sdk::igris_c_sdk)
```

Python wheel — quickest

```
python3 -m venv .venv
source .venv/bin/activate
pip install dist/igris_c_sdk-*.whl
```

Python wheel — reproducible env via pixi

```
# Install pixi once: https://pixi.sh/latest/
cd python
pixi install
pixi run pip install ../dist/igris_c_sdk-*.whl
```

pixi shell # activate the env

ROS 2 (colcon workspace)

Build this from a shell with no Python environment active — leave the pixi shell
or venv from the section above first. CMake picks an interpreter from $CONDA_PREFIX /
$VIRTUAL_ENV before the system one, and ROS 2's generators need the system Python.

Copy ros2/src/igris_c_sdk_public/ — the package, not the tree root of the
same name — into your own workspace src/. The package's declared name
(igris_c_sdk) matches what the SDK bridge publishes, so DDS type ids
interoperate without renaming.

```
cp -r ros2/src/igris_c_sdk_public ~/my_ws/src/
cd ~/my_ws && colcon build
source install/setup.bash
ROS_DOMAIN_ID=5 ros2 run igris_c_sdk ros_lowstate_listener \
--ros-args -p robot_namespace:=igris_c_〈your_robot_id〉
```

If nothing appears, discovery is not reaching the robot (see
When discovery still finds nothing). Add the
profile and the same command works — this is the full invocation, nothing else is
needed:

```
FASTRTPS_DEFAULT_PROFILES_FILE=$HOME/.config/igris_c/fastdds_peers.xml \
ROS_DOMAIN_ID=5 \
ros2 run igris_c_sdk ros_lowstate_listener --ros-args -p robot_namespace:=igris_c_〈your_robot_id〉
```

See ros2/README.md for the full ROS 2 walkthrough,
including using the SDK types from your own nodes.

The first arg to the C++ examples is the DDS domain id (mirrors
ROS_DOMAIN_ID); the namespace prefix matches the robot's bridge
config user_suffix.

Examples

All examples take the DDS domain id and robot namespace (C++/ROS 2 as args, Python via
--domain-id /
--namespace). Set CYCLONEDDS_URI first (see Connecting to the robot).

Each row is a capability; the cell shows the example for that interface, or — if not provided.

| Capability | C++ (cyclonedds/) | Python (python/) | ROS 2 (ros2/) |
| --- | --- | --- | --- |
| Low-level control (LowCmd/LowState) | cyclonedds_low_level_control | client.py | |

ros_lowcmd_publisher, ros_lowstate_listener

Service API (init / torque / control-mode)

cyclonedds_service

client.py

ros_service

End-effector (hand)

cyclonedds_hand_example

hand_example.py

ros_hand_example, ros_hand_gui_client

Walk velocity (Twist)

cyclonedds_twist_publisher

twist_publisher.py

ros_twist_publisher

GUI client

cyclonedds_gui_client, cyclonedds_hand_gui_client

sdk_gui_client.py, sdk_hand_gui_client.py

ros_gui_client

Sensor viewer

cyclonedds_sensor_viewer [1]

sensor_viewer.py

ros_sensor_listener, ros_sensor_viewer

Notebook walkthrough

—

client.ipynb

—

[1] built only when OpenCV is present.

API reference (summary)

The SDK communicates over DDS in two patterns: Publish/Subscribe (continuous state/command streams) and

Request/Response (services). All topics resolve under the robot namespace described above.

High-level services — IgrisC_Client

Function

Method

Service topic

PDU / motor init

InitPdu(uint8_t init_type, timeout_ms)

service/pdu_init

Torque on/off

SetTorque(uint8_t torque, timeout_ms)

service/torque

End-effector (hand) init

InitHand(timeout_ms)

service/hand_init

Control-mode switch

SendControlModeCommand(type, preset_id, is_cyclic, timeout_ms)

service/control_mode

Each service is a 〈topic〉/request + 〈topic〉/response pair, matched by request_id; responses carry success / message / error_code (0 = success).

This pairing is how the cyclonedds lane serves services, under either topic layout — it is not a ROS 2 service and ros2 service list will not show it. On transport: ros the robot serves real rclcpp services instead; use ros2/'s ros_service there. Pick the lane first, then the example for that lane.

Constants are uint8 values on the request message, not enum types. Reach them through the generated constant namespaces:

PduInitCmd_Request_Constants: PDU_INIT_NONE (power OFF) · PDU_INIT (power ON) · MOTOR_INIT · PDU_AND_MOTOR_INIT · PDU_OFF (power OFF, same as PDU_INIT_NONE)

TorqueCmd_Request_Constants: TORQUE_NONE (reserved) · TORQUE_ON · TORQUE_OFF

Control modes — ControlModeCommandRequest_Request_Constants

MOTION_PRESET (needs preset_id; HIGH_LEVEL only) · MOTION_PRESET_CYCLIC_TOGGLE · JOINT_POSITION_HOLD ·

MOTION_STOP (HIGH_LEVEL only) · WALKMODE_ON · LOW_LEVEL_JOINT_CONTROL · LOW_LEVEL_WALKMODE_ON ·

HIGH_LEVEL_JOINT_CONTROL · HIGH_LEVEL_WALKMODE_ON. (CUSTOM_MODE_* are internal — not for SDK use.)

Cyclic repeat of a preset is typically requested via the is_cyclic flag on MOTION_PRESET (as the examples do);

MOTION_PRESET_CYCLIC_TOGGLE is the dedicated toggle command.

A mode switch is accepted only when body power + torque are ON and E-Stop is released; it is blocked while

the robot is in the stopped state (run the HOME motion preset to exit).

Low-level control

Stream targets on lowcmd (LowCmd) and read state on lowstate (LowState). MotorCmd carries q/dq/tau/kp/kd per joint (31 DOF, HYBRID mode); LowCmd.kinematic_modes[5] selects MS/PJS per

parallel-link group. See the cyclonedds/ and python/ low-level examples.

End effector (hand)

A single DDS interface (handcmd / handstate / service/hand_init) drives whichever end effector

the robot is built with — 5-finger hand (ids 11–16 right / 21–26 left), 1-DoF gripper, or magnet.

HandCmd.motor_cmd[].q is normalized 0–1. Initialize via InitHand() (or a service/hand_init request).

License

The SDK source is licensed under the terms in cyclonedds/LICENSE. This release bundles Cyclone DDS, Fast CDR, and foonathan_memory as static libraries — see cyclonedds/licenses/ and cyclonedds/NOTICE for upstream license texts.

About

IGRIS_C SDK

Resources

Readme

Activity

Custom properties

Stars

4 stars

Watchers

0 watching

Forks

2 forks

Report repository

Releases

Packages

Contributors

Languages